

Date: _____

Lab # 02 TO IMPLEMENT CONDITIONAL STATEMENTS AND LOOPS IN PYTHON

Objective:

- Understand and apply basic control structures such as if, else, while, and for.
- Learn to use nested loops and nested conditions.
- Practice writing efficient programs using these control structures.

Lab Theory: Control Structures

Control structures are essential elements in programming that allow developers to dictate the flow of execution within a program. These structures enable the program to make decisions, repeat tasks, or perform complex logic based on certain conditions.

1. Conditional Statements: if, else, elif

Conditional statements allow a program to take different paths based on certain conditions. In Python (or other similar languages), these statements include if, else, and elif.

1.1 if Statement:

The if statement evaluates a condition and, if it is true, executes a block of code.

Syntax:

```
python
Copy code
if condition:
    # Code to execute if the condition is True
```

Example:

```
python
Copy code
x = 5
if x > 0:
    print("x is positive")
```

1.2 else Statement:

The else statement executes a block of code if the if condition is False.

Syntax:

```
python
Copy code
```

```
if condition:
    # Code if condition is True
else:
    # Code if condition is False
```

Example:

```
python
Copy code
x = -3
if x > 0:
    print("x is positive")
else:
    print("x is negative or zero")
```

1.3 elif Statement:

The elif (short for "else if") statement checks multiple conditions, one after the other.

Syntax:

```
python
Copy code
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition1 is False and condition2 is True
else:
    # Code if all previous conditions are False
```

Example:

```
python
Copy code
x = 10
if x > 10:
    print("x is greater than 10")
elif x == 10:
    print("x is equal to 10")
else:
    print("x is less than 10")
```

2. Loops: while and for

Loops allow a program to repeat a block of code multiple times, either for a specific number of iterations or until a condition is met.

2.1 while Loop:

A while loop repeats a block of code as long as a condition is True.

Syntax:

```
python
Copy code
while condition:
    # Code to execute while the condition is True
```

Example:

```
python
Copy code
i = 1
while i <= 5:
    print(i)
    i += 1
```

Explanation: This loop will print the numbers 1 through 5. The loop continues until i becomes greater than 5.

2.2 for Loop:

A for loop is used to iterate over a sequence (like a list, string, range, etc.).

Syntax:

```
python
Copy code
for variable in sequence:
    # Code to execute for each item in the sequence
```

Example:

```
python
Copy code
for i in range(1, 6):
    print(i)
```

Explanation: This loop will print the numbers 1 through 5, where the range(1, 6) function generates numbers from 1 to 5.

3. Nested Loops and Conditional Statements

3.1 Nested for Loops:

Nested loops are loops inside other loops. They allow for more complex iterations, such as printing patterns or working with multidimensional data.

Syntax:

```
python
Copy code
for variable1 in sequence1:
```

```
for variable2 in sequence2:  
    # Code to execute for each combination of items in sequence1 and sequence2
```

Example:

```
python  
Copy code  
for i in range(1, 4):  
    for j in range(1, 4):  
        print(f'i = {i}, j = {j}')
```

Explanation: This code will produce a combination of values of i and j, with the outer loop iterating over i and the inner loop iterating over j.

3.2 Nested if Statements:

Nested if statements are conditional statements placed inside other if statements, allowing for more specific conditions to be checked after an initial condition.

Syntax:

```
python  
Copy code  
if condition1:  
    if condition2:  
        # Code to execute if both condition1 and condition2 are True
```

Example:

```
python  
Copy code  
x = 5  
y = 10  
  
if x > 0:  
    if y > 5:  
        print("x is positive and y is greater than 5")
```

Explanation: The program checks both conditions: whether x is greater than 0 and whether y is greater than 5. If both conditions are true, the print statement is executed.

4. Break and Continue Statements

4.1 break Statement:

The break statement exits a loop prematurely when a condition is met.

Example:

```
python
Copy code
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

Explanation: The loop will print numbers 1 and 2, but when i becomes 3, the break statement is triggered, and the loop ends.

4.2 continue Statement:

The continue statement skips the current iteration of the loop and moves to the next iteration.

Example:

```
python
Copy code
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Explanation: The loop will print the numbers 1, 2, 4, and 5. When i is 3, the continue statement skips that iteration and moves to the next.

5. Use Cases for Control Structures

- **Decision-making:** Conditional statements are useful when different actions need to be taken based on input or specific conditions, like deciding whether a user qualifies for a loan or not.
- **Repetition:** Loops are commonly used for tasks that need to be repeated, such as processing every element in a list, generating sequences, or handling user input validation.
- **Nested logic:** Nested loops and conditionals are helpful when dealing with multi-dimensional data (such as matrices), building complex patterns, or handling scenarios where multiple layers of decisions are required.

6. Conclusion

Control structures such as if-else statements, while and for loops, and their nested forms are the foundation of flow control in programming. They allow developers to implement decision-making processes, repeat tasks, and handle complex logic efficiently. Understanding and mastering these control structures is key to writing efficient and effective code.

1. Write a program that asks the user to enter a number. If the number is positive, print "Positive". If the number is negative, print "Negative". If the number is zero, print "Zero".
2. Write a program that takes a student's marks as input. Use nested if statements to assign a grade:
 - a $\geq 90 \rightarrow A$
 - b ≥ 75 and $< 90 \rightarrow B$
 - c ≥ 50 and $< 75 \rightarrow C$
 - d $< 50 \rightarrow F$

- ```
*
**


```

- [illegible]

- Page 9